# P2591R4
# Concatenation of strings and string views

Published Proposal, 2023-06-08

**Author:**
Giuseppe D'Angelo

**Audience:**
LEWG

**Project:**
ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

---

## Abstract

We propose to add overloads of `operator+` between string and string view classes.

## Table of Contents

# § 1. Changelog

- R4
  - Incorporated feedback from the 2023-04-18 LEWG telecon.
  - Reverted to a free function template implementation (Approach 2, as described below), following the poll(s) that showed no longer a consensus for using hidden friends and thus introducing an asymmetry.
  - Added the requested tests (involving `std::filesystem::path` and a test producing ambiguities) to the working prototype.
  - Rebased the wording on top of the latest draft.
- R3
  - Reading improvements.
- R2
  - Changed the signatures of the proposed operators to take precisely string views (instead of anything convertible to string views).
  - Made the proposed operators hidden friends, following a LEWG vote in a telecon.
  - Added a discussion about the implications of such a change.
  - Added an Annex C entry.

- R1
  - Minor reading improvements.
- R0
  - First submission.

## § 2. Motivation and Scope

The Standard is currently lacking support for concatenating strings and string views by means of `operator+`:

```
std::string calculate(std::string_view prefix)
{
  return prefix + get_string(); // ERROR
}
```

This constitutes a major asymmetry when considering the rest of `basic_string`'s API related to string concatenation. In such APIs there is already support for the corresponding view classes.

In general, this makes the concatenation APIs between string and string views have a poor usability experience:

```
std::string str;
std::string_view view;

// Appending
str + view;               // ERROR
str + std::string(view); // OK, but inefficient
str + view.data();        // Compiles, but BUG!

std::string copy = str;
copy += view;             // OK, but tedious to write (requires explicit cop
copy.append(view);        // OK, ditto


// Prepending
view + str;               // ERROR
```

```
std::string copy = str;
copy.insert(0, view);    // OK, but tedious and inefficient
```

Similarly, the current situation is asymmetric when considering concatenation against raw pointers:

```
std::string str;

str + "hello";    // OK
str + "hello"sv;  // ERROR

"hello"   + str;  // OK
"hello"sv + str;  // ERROR
```

All of this is just bad ergonomics; the lack of `operator+` is extremely surprising for end-users (cf. this StackOverflow question), and harms teachability and usability of `string_view` in lieu of raw pointers.

Now, as shown above, there *are* workarounds available either in terms of named functions (`append`, `insert`, ...) or explicit conversions. However it's hard to steer users away from the convenience syntax (which is ultimately the point of using `operator+` in the first place). The availability of the *other* overloads of `operator+` opens the door to bad code; for instance, it risks neglecting the value of view classes:

```
std::string prepend(std::string_view prefix)
{
  return std::string(prefix) + get_string(); // inefficient
}
```

And it may even open the door to (subtle) bugs:

```
std::string result1 = str + view; // ERROR. <Sigh>, ok, let me rewrite as..

std::string result2 = str + std::string(view); // OK, but this is inefficie

std::string result3 = str + view.data(); // Compiles; but BUG!
```

The last line exhibits undefined behavior if `view` is not NUL terminated, and also behaves differently in case it has embedded NULs.

This paper proposes to fix these API flaws by adding suitable `operator+` overloads between string and string view classes. The changes required for such operators are straightforward and should pose no burden on implementations.

## § 2.1. Why are those overloads missing in the first place?

[N3685] ("`string_view`: a non-owning reference to a string, revision 4") offers the reason:

> I also omitted `operator+(basic_string, basic_string_view)` because LLVM returns a lightweight object from this overload and only performs the concatenation lazily. If we define this overload, we'll have a hard time introducing that lightweight concatenation later.

Subsequent revisions of the paper no longer have this paragraph.

There is a couple of considerations that we think are important here.

- `string_view` has been approved for C++17 in Jacksonville (February 2016). At the time of this writing, such a "string builder" facility has not been proposed for standardization (as far as we know). Neglecting a completely reasonable feature to users (concatenation via `operator+`) for so long, in the name of an yet unseen future "major" feature, is a disservice to them.
- We strongly feel that overloading `operator+` is completely outside of the design space for a string builder class. There is absolutely no reason why `str + "hello"sv` should use the builder, but `str + "hello"` should not -- not to mention cases like `strA + strB + strC`. One cannot however change the semantics of the existing `operator+` overloads without breaking API/ABI compatibility. In Qt, [QStringBuilder] uses `operator%` by default; blindly replacing `operator+` with `operator%` when concatenating strings comes with its own share of problems (not only it is API incompatible, but it causes dangling references in a number of scenarios).

In short: we do not see any reason to further withhold the proposed additions.

# § 3. Impact On The Standard

This proposal is a pure library extension.

This proposal does not depend on any other library extensions.

This proposal does not require any changes in the core language.

# 4. Design Decisions

## 4.1. Minimizing the number of allocations

The proposed wording builds on top / reuses of the existing one for `CharT *`. In particular, no attempts have been made at e.g. minimizing memory allocations (by allocating only one buffer of suitable size, then concatenating in that buffer). Implementations already employ such mechanisms internally, and we would expect them to do the same also for the new overloads (for instance, see here for libstdc++ and here for libc++).

## 4.2. Should the proposed operators be hidden friends? Should they be function templates?

There are several ways to define the proposed overloads.

### 4.2.1. Approach 1: free non-friend function templates, taking exactly a string view

The signature would look like this:

```
template<class charT, class traits = char_traits<charT>,
         class Allocator = allocator<charT>>
  class basic_string {
    // [...]
  };

template<class charT, class traits, class Allocator>
constexpr basic_string<charT, traits, Allocator>
  operator+(const basic_string<charT, traits, Allocator>& lhs,
            basic_string_view<charT, traits> rhs);
// Repeat for the other overloads with swapped arguments, rvalues, etc.
```

This approach closely follows the pre-existing overloads for `operator+`. In particular, here the newly added operators are not hidden friends (which may increase compilation times, give worse compile errors, etc.).

Still: just like hidden friends, it is **not** possible to use these operator with datatypes implicitly convertible to `std::basic_string` / `std::basic_string_view` specializations:

```
class convertible_to_string
{
public:
  /* implicit */ operator std::string() const;
};

convertible_to_string cts;

cts + "hello"s;    // ERROR (pre-existing)
cts + "hello"sv;   // ERROR
```

The error stems from the fact that the existing (and the proposed) `operator+` are function templates, and implicit conversions are not possible given the signatures of these functions: all the parameter types of `operator+` contain a *template-parameter* that needs to be deduced, in which case implicit conversions are not considered (this is [temp.arg.explicit/7]).

While the lack of support for types implictly convertible to strings may be desiderable (for symmetry), the lack of support for types implictly convertible to string *views* is questionable. String view operations explictly support objects of types convertible to them. For instance:

```
std::string s;
convertible_to_string cts;

s == cts;   // ERROR


std::string_view sv;
convertible_to_string_view ctsv;

sv == ctsv; // OK; [string.view.comparison/1]
```

The above definition of the overloads would prevent types convertible to string views to be appended/prepended to strings, again because the implicit conversion towards the string view type would be prevented. This would even be inconsistent with the existing string's member functions:

```
std::string s;
convertible_to_string_view ctsv;

s.append(ctsv); // OK, [string.append/3]
s + ctsv;       // ERROR, ???
```

Finally, overloads added as non-member/non-friend function templates are not viable when using something like `std::reference_wrapper`:

```
std::reference_wrapper<std::string> rs(~~~);
std::reference_wrapper<std::string_view> rsv(~~~);

rs + rs;  // ERROR (pre-existing)
rs + rsv; // ERROR
```

This is because an argument of type e.g. `std::reference_wrapper<std::string>` (i.e. `std::reference_wrapper<std::basic_string<char, std::char_traits<char>, std::allocator<char>>>`) can never match against a parameter of type `std::basic_string<charT, traits, Allocator>`.

### § 4.2.2. Approach 2: free non-friend function templates, taking anything convertible to a string view

This is similar to approach n. 1, except that the string view argument would also accept any type which is convertible to a string view. The precedent for this would be the existing functions for concatenating/inserting strings (e.g. `append`, `insert`, `operator+=`), all of which take a parameter of any type *convertible* to a string view; as well as the comparison operators for string views, where "[...] implementations shall provide sufficient additional overloads [...] so that an object `t` with an implicit conversion to `S` can be compared" ([string.view.comparison/1]).

Therefore, the proposed signatures would look like this:

```
template<class charT, class traits = char_traits<charT>,
         class Allocator = allocator<charT>>
  class basic_string {
```

```
    // [...]
  };

template<class charT, class traits, class Allocator>
constexpr basic_string<charT, traits, Allocator>
  operator+(const basic_string<charT, traits, Allocator>& lhs,
            type_identity_t<basic_string_view<charT, traits>> rhs);
  //        ^^^^^^^^^^^^^^^^ make a non-deduced context


// Repeat for the other overloads with swapped arguments, rvalues, etc.
```

NOTE: this may or may not be the actual *proposed wording*. We could instead handwave the actual overload set by using the "sufficient additional overloads" wording. An implementation could therefore choose to use another implementation strategy, such as SFINAE, constraints, and so on. (See also [LWG3950].)

Apart from allowing to concatenate strings with objects of types *convertible* to string views, this approach still forbids the usage of types convertible to strings, as well as types such as `reference_wrapper`:

```
std::string s;
std::string_view sv;
convertible_to_string cts;
convertible_to_string_view ctsv;

s + sv;    // OK
s + ctsv;  // OK

s + cts;   // ERROR
cts + sv;  // ERROR
```

### § 4.2.3. Approach 3: hidden friends, non-template functions

Basically, this would be an application of the Barton–Nackman idiom in combination with hidden friends ([hidden.friends]).

The proposed operators would look like this:

```
template<class charT, class traits = char_traits<charT>,
         class Allocator = allocator<charT>>
  class basic_string {
    // [...]

    constexpr friend basic_string
      operator+(const basic_string& lhs,
                basic_string_view<charT, traits>) { /* hidden friend */ }
    // Repeat for the other overloads with swapped arguments, rvalues, etc.
  };
```

In such an approach, one of the arguments must still be a string object, otherwise the overload is not even added to the overload set (hidden friend).

The other argument can be any object implicitly convertible to a string view. Since the overload is not a function template, implicit conversions here "kick in" and work as expected, without the need of adding additional overloads (or declaring the operators as function templates):

```
std::string s;
convertible_to_string_view ctsv;

s + ctsv;  // OK
```

**There is a perhaps surprising side-effect**, however: defining this overload set would also allow concatenation between a string and an object convertible to a *string*. For instance:

```
std::string s;
convertible_to_string cts;

s == cts;  // ERROR
s +  cts;  // OK (!)
```

In the last line, the lhs of type `std::string` makes the various `operator+(std::string, std::string_view)` overloads visible to lookup. Then, the `operator+(std::string_view, std::string&&)` is selected, converting the `lhs` from `std::string` to `std::string_view` and the `rhs` from `convertible_to_string` to a rvalue `std::string`.

Finally, using types such as `std::reference_wrapper` would work transparently:

```
std::reference_wrapper<std::string> rs(~~~);

rs + "hello"sv; // OK
```

In this example, ADL would add the hidden friend operators to the overload set (cf. [basic.lookup.argdep/3.2]), operators which again are non-template functions. Then, the `operator+(const std::string &, std::string_view)` is selected, since we can implicitly convert the first parameter from the argument of type `std::reference_wrapper<std::string>`.

### 4.2.4. Approach 4: hidden friends, function templates, taking anything convertible to a string view

This approach is similar to approach 2, however makes the proposed operators hidden friends.

The proposed operators would in principle look like this:

```
template<class charT, class traits = char_traits<charT>,
         class Allocator = allocator<charT>>
  class basic_string {
    // [...]

    template <class C, class T, class A>
      constexpr friend basic_string<C, T, A>
        operator+(const basic_string<C, T, A>& lhs,
                  basic_string_view<C, T>) { /* hidden friend */ }

    template <class C, class T, class A>
      constexpr friend basic_string<C, T, A>
        operator+(const basic_string<C, T, A>& lhs,
                  type_identity_t<basic_string_view<C, T>>) { /* hidden fri

    // Repeat for the other overloads with swapped arguments, rvalues, etc.
  };
```

In practice the above does *not work*, as it leads to redefinition errors if `basic_string` is instatiated with different template parameters (which is of course the case). Note that, in order for the operators to

be hidden friends, their *definition* must be present in `basic_string`'s class body; multiple instantiations of `basic_string` would therefore redefine the same function template multiple times.

Hence, an actual implementation has to employ some tricks, such as isolating the operators in a non-template base class:

```
template<class charT, class traits, class Allocator>
  class basic_string;

class __basic_string_base // exposition-only
{
  template <class C, class T, class A>
    constexpr friend basic_string<C, T, A>
      operator+(const basic_string<C, T, A>& lhs,
                basic_string_view<C, T>) { /* hidden friend */ }

  template <class C, class T, class A>
    constexpr friend basic_string<C, T, A>
      operator+(const basic_string<C, T, A>& lhs,
                type_identity_t<basic_string_view<C, T>>) { /* hidden frier

  // Repeat for the other overloads with swapped arguments, rvalues, etc.
};

template<class charT, class traits = char_traits<charT>,
         class Allocator = allocator<charT>>
  class basic_string : __basic_string_base {
    // [...]
  };
```

This approach brings the same semantics as of approach 2, with the exception that the operators are not found through ordinary unqualified/qualified lookup (because they are hidden friends). It is still not possible to call these operators using an argument of a type convertible to string, nor to call them through `reference_wrapper`.

### § 4.2.5. Summary

| Works between...? | Approach 1 | Approach 2 | Approach 3 | Approach 4 |
|---|---|---|---|---|
| `std::string` and `std::string_view` | ✔ | ✔ | ✔ | ✔ |

| Works between...? | Approach 1 | Approach 2 | Approach 3 | Approach 4 |
|---|---|---|---|---|
| `std::string` and an object convertible to `std::string_view` | ✘ | ✔ | ✔ | ✔ |
| `std::string` and an object convertible to `std::string` | ✘ | ✘ | ✔ | ✔ |
| Two objects convertible to `std::string` | ✘ | ✘ | ✘ | ✘ |
| `std::reference_wrapper<std::string>` and `std::string_view` | ✘ | ✘ | ✔ | ✘ |

#### 4.2.6. Which strategy should be used?

The R1 revision of this paper implemented approach 2, for symmetry with the pre-existing overloads of `operator+` between strings.

During the 2022-08-16 LEWG telecon, a poll indicated weak consensus (2/5/4/2/0) for making the proposed operators hidden friends, even at the cost of making them inconsistent with the existing overloads.

R3 of this paper implemented approach 3, and elaborated on the consequences of the different approaches, including the (possibly unexpected) ability of concatenating objects of types convertible to *strings*. During the review of R3 in the LEWG telecon on 2023-04-18, when presented with this information, there was no longer consensus for the hidden friends approach.

Therefore, in R4 we are reverting to approach 2, again with the idea of keeping the overload set consistent with the pre-existing overloads.

## 4.3. Backwards compatibility and Annex C

Library Evolution has requested a note to be added to Annex C in case the proposed operators break backwards compatibility.

If users define an `operator+` overload between classes from the Standard Library (in another namespace than `std`), and then the Standard Library starts providing such an overload and user code stops compiling (due to redefinitions, ambiguities, etc.), does this constitute a source-incompatible change?

[SD-8] is not particularly explicit on the subject of adding new overloads for *operators*, although it does state that:

> Primarily, the standard reserves the right to:
>
> [...]
>
> - Add new names to any entity within any reserved namespace, including but not limited to:
>
>   - Functions (this includes new member functions and overloads to existing functions)

Operators are functions, but they're also a particular class of them, as they are practically never called using an explicit function-call expression. Instead, any ordinary code relies on the special rules of overload resolution for operators ([over.match.oper]).

The question here is therefore is whether the Standard Library is simply allowed to alter the overload set available to operators, when they are used on objects of datatypes defined in the library itself. It is easy to argue that, if *both* arguments to an operator overload are library datatypes, then the library reserves the right to add such overload without worrying about any possible breakage. Implicit conversions and ADL make however the situation slightly more complex.

We can construct an example as follows. The proposed `operator+` overloads require one of the arguments to be an object of a `std::basic_string` specialization (see the discussion above regarding [temp.arg.explicit/7]). Let's therefore focus on the *other* argument's type.

Suppose that a user declared a `operator+` overload like this:

```
struct user_datatype;

R operator+(std::string, user_datatype);
```

Then this overload will always be preferred to the ones that we are proposing (when passing a parameter of type `user_datatype`). This works even if the type is implictly convertible to `std::string_view` and therefore overload resolution does not exclude the overloads of the present proposal:

```
struct convertible_to_string_view
{
  /* implicit */ operator std::string_view() const;
};
R operator+(std::string, convertible_to_string_view); // pre-existing
```

```
convertible_to_string_view ctsv;

"hello"s + ctsv; // still calls the user-defined operator+, as it's a bette
```

Let's furthermore consider a further type convertible to both a user-defined datatype as well as `std::string_view`. This could be, for instance, a type convertible to a pre-C++17 custom string view class which has also been "modernized" by adding a conversion to `std::string_view`:

```
struct my_string_view; // pre-c++17, legacy
std::string operator+(std::string, my_string_view);
```

```
struct char_buffer
{
  /* implicit */ operator my_string_view() const;   // legacy
  /* implicit */ operator std::string_view() const; // modern
};


char_buffer buf;
std::string result = "hello"s + buf; // OK
```

Although it may seem that the call to `operator+` would now be ambiguous between `operator+(std::string, my_string_view)` and `operator+(std::string, std::string_view)`, it actually is *not ambiguous* and even calls the *pre-existing* `operator+` taking a `my_string_view`. The reason for this is that although both `operator+` overloads are viable, the one taking a `my_string_view` as defined above is not a function template, while the one taking a `std::string_view` is actually a function template specialization; the former overload ranks better ([over.match.best.general]/2.4).

What if the user-defined `operator+` is itself a function template? For instance:

```
template <typename Char>
struct basic_my_string_view;

using my_string_view = basic_my_string_view<char>;

template <typename Char>
  std::basic_string<Char>
    operator+(std::basic_string<Char>, basic_my_string_view<Char>);
```

```
struct char_buffer
{
  /* implicit */ operator my_string_view() const;
  /* implicit */ operator std::string_view() const;
};


char_buffer buf;
std::string result = "hello"s + buf; // was: ERROR; with the proposed chang
```

The above code does not compile without the changes introduced by this paper, again because implicit conversions are not considered due to the deducible `Char` template parameter. With the changes introduced by this paper, the code now compiles, and `operator+(std::string, std::string_view)` is called; the pre-existing overload is still not viable. In other words: in this specific scenario the impact is positive.

What if user code employed some technique to enable implicit conversions with `operator+`, for instance like this:

```
template <typename Char>
struct basic_my_string_view;

using my_string_view = basic_my_string_view<char>;

template <typename Char>
  std::basic_string<Char>
    operator+(std::basic_string<Char>, std::type_identity_t<basic_my_string

struct char_buffer
{
  /* implicit */ operator my_string_view() const;
  /* implicit */ operator std::string_view() const;
};


char_buffer buf;
std::string result = "hello"s + buf; // was: OK; with the proposed changes:
```

In this last scenario the call to `operator+` becomes ambiguous with the proposed changes.

[SD-8] does not seem to offer guidance here: is it OK for the Standard Library to break code that is "too generous" in its implicit conversions? In case, we are going to stay on the safe side, and document this possible breakage in Annex C.

## § 5. Implementation experience

A working prototype of the changes proposed by this paper, done on top of GCC 13.1, is available in this GCC branch on GitHub. The entire libstdc++ testsuite passes with the changes applied. A smoke test is included.

Will Hawkins has very kindly contributed an implementation in libc++.

## § 6. Technical Specifications

All the proposed changes are relative to [N4950].

### § 6.1. Feature testing macro

In [version.syn], modify

```
#define __cpp_lib_string_view 201803L YYYYMML // also in <string>, <string_v
```

with the value specified as usual (year and month of adoption of the present proposal).

## § 6.2. Proposed wording

Modify [string.syn] as shown:

```
namespace std {
    [...]

  template<class charT, class traits, class Allocator>
    constexpr basic_string<charT, traits, Allocator>
      operator+(basic_string<charT, traits, Allocator>&& lhs,
                charT rhs);
  template<class charT, class traits, class Allocator>
    constexpr basic_string<charT, traits, Allocator>
      operator+(const basic_string<charT, traits, Allocator>& lhs,
                basic_string_view<charT, traits> rhs);
  template<class charT, class traits, class Allocator>
    constexpr basic_string<charT, traits, Allocator>
      operator+(basic_string<charT, traits, Allocator>&& lhs,
                basic_string_view<charT, traits> rhs);
  template<class charT, class traits, class Allocator>
    constexpr basic_string<charT, traits, Allocator>
      operator+(basic_string_view<charT, traits> lhs,
                const basic_string<charT, traits, Allocator>& rhs);
  template<class charT, class traits, class Allocator>
    constexpr basic_string<charT, traits, Allocator>
      operator+(basic_string_view<charT, traits> lhs,
                basic_string<charT, traits, Allocator>&& rhs);
  // see [string.op.plus.string_view], sufficient additional overloads of


  template<class charT, class traits, class Allocator>
    constexpr bool
      operator==(const basic_string<charT, traits, Allocator>& lhs,
                 const basic_string<charT, traits, Allocator>& rhs) noexc
```

✧ ✧ ✧

**Append** the following content at the end of [string.op.plus]:

```
template<class charT, class traits, class Allocator>
  constexpr basic_string<charT, traits, Allocator>
    operator+(const basic_string<charT, traits, Allocator>& lhs,
              basic_string_view<charT, traits> rhs);
```

— *Effects*: Equivalent to:

```
basic_string<charT, traits, Allocator> r = lhs;
r.append(rhs);
return r;
```

```
template<class charT, class traits, class Allocator>
  constexpr basic_string<charT, traits, Allocator>
    operator+(basic_string<charT, traits, Allocator>&& lhs,
              basic_string_view<charT, traits> rhs);
```

— *Effects*: Equivalent to:

```
lhs.append(rhs);
return std::move(lhs);
```

```
template<class charT, class traits, class Allocator>
  constexpr basic_string<charT, traits, Allocator>
    operator+(basic_string_view<charT, traits> lhs,
              const basic_string<charT, traits, Allocator>& rhs);
```

— *Effects*: Equivalent to:

```
basic_string<charT, traits, Allocator> r = rhs;
r.insert(0, lhs);
return r;
```

```
template<class charT, class traits, class Allocator>
  constexpr basic_string<charT, traits, Allocator>
    operator+(basic_string_view<charT, traits> lhs,
              basic_string<charT, traits, Allocator>&& rhs);
```

> 🔲 *Effects*: Equivalent to:
>
> ```
> rhs.insert(0, lhs);
> return std::move(rhs);
> ```

✧ ✧ ✧

Add a new subclause after [string.op.plus] with the following content:

> **🔲 Concatenation of strings and string views [string.op.plus.string_view]**
>
> 1) Let `S` be `basic_string<charT, traits, Allocator>`; let `s` be an instance of `S`; and let `SV` be `basic_string_view<charT, traits>`. Implementations shall provide sufficient additional overloads marked `constexpr` so that an object `t` with an implicit conversion to `SV` can be concatenated with `s` according to Table 🔲.
>
> *Table 🔲: Additional `basic_string` concatenation overloads [tab:string.op.plus.string_view.overloads]*
>
| Expression | Equivalent to |
|---|---|
| `s + t` | `s + SV(t)` |
| `t + s` | `SV(t) + s` |
| `std::move(s) + t` | `std::move(s) + SV(t)` |
| `t + std::move(s)` | `SV(t) + std::move(s)` |

✧ ✧ ✧

In [diff], add a new subclause, tentatively named **[diff.cpp26.strings]**.

**Note to the editor**: such a subclause should be under [diff.cpp26], which by the time this proposal is adopted, may or may not exist yet. The naming and contents of the parent [diff.cpp26] subclause should match the existing ones (e.g. [diff.cpp20]), of course adapted to C++26.

> **C.🔲.🔲 [strings]: strings library [diff.cpp26.strings]**

(1) **Affected subclause**: [string.classes]

**Change**: Additional overloads of `operator+` between `basic_string` specializations and types convertible to `basic_string_view` specializations have been added.

**Rationale**: Make `operator+` consistent with the existing overloads.

**Effect on original feature**: Valid C++23 code may fail to compile in this revision of C++. For instance:

```
template <typename Char>
struct basic_my_string_view;

using my_string_view = basic_my_string_view<char>;

template <typename Char>
  std::basic_string<Char>
    operator+(std::basic_string<Char>, std::type_identity_t<basic_my_stri

struct char_buffer
{
  operator my_string_view() const;
  operator std::string_view() const;
};

int main() {
  char_buffer buf;
  std::string result = std::string("hello") + buf; // ill-formed (ambiguc
}
```

## 7. Acknowledgements

Thanks to KDAB for supporting this work.

Thanks to Will Hawkins for the discussions and the prototype implementation in libc++.

All remaining errors are ours and ours only.

# § References

## § Informative References


**[Clazy-qstringbuilder]**

*auto-unexpected-qstringbuilder*. URL: https://github.com/KDE/clazy/blob/master/docs/checks/README-auto-unexpected-qstringbuilder.md

**[Libcpp-string-concatenation]**

*operator+ between pointer and string in libc++*. URL: https://github.com/llvm/llvm-project/blob/llvmorg-14.0.0/libcxx/include/string#L4218

**[Libstdcpp-string-concatenation]**

*operator+ between pointer and string in libstdc++*. URL: https://github.com/gcc-mirror/gcc/blob/releases/gcc-12.1.0/libstdc%2B%2B-v3/include/bits/basic_string.tcc#L603

**[LWG3950]**

Giuseppe D'Angelo. *std::basic_string_view comparison operators are overspecified*. New. URL: https://wg21.link/lwg3950

**[N3685]**

Jeffrey Yasskin. *string_view: a non-owning reference to a string, revision 4*. 3 May 2013. URL: https://wg21.link/n3685

**[N4950]**

Thomas Köppe. *Working Draft, Standard for Programming Language C++*. 10 May 2023. URL: https://wg21.link/n4950

**[P2591-GCC]**

Giuseppe D'Angelo. *P2591 prototype implementation for libstdc++*. URL: https://github.com/dangelog/gcc/tree/P2591_string_view_concatenation

**[P2591-LLVM]**

Will Hawkins. *P2591 prototype implementation for libc++*. URL: https://github.com/hawkinsw/llvm-project/tree/P2591_string_view_concatenation

**[QStringBuilder]**

*QStringBuilder documentation*. URL: https://doc.qt.io/qt-6/qstring.html#more-efficient-string-construction

**[SD-8]**

Titus Winter. *SD-8: Standard Library Compatibility*. URL: https://isocpp.org/std/standing-documents/sd-8-standard-library-compatibility

**[StackOverflow]**

*Why is there no support for concatenating std::string and std::string_view?*. URL: https://stackoverflow.com/questions/44636549/why-is-there-no-support-for-concatenating-

stdstring-and-stdstring-view